

算法 `airfogsim_algorithm.py`

`airfogsim_algorithm.py` 文件实现了 AirFogSim 平台的算法模块，主要包含两个类：基础算法模块 `BaseAlgorithmModule` 和一个扩展算法 `NVHAUAlgorithmModule`（Nearest Vehicle and Highest Accuracy UAV）。这些模块负责与环境交互，制定各种调度决策。

整体架构

该文件提供了一个算法框架，使用不同的调度器与环境交互。主要通过以下几个方面的调度决策来控制仿真过程：

- 任务卸载（Offloading）
- 计算资源分配（Computing）
- 通信资源分配（Communication）
- 任务分配（Mission）
- 任务结果返回路由（Returning）
- 交通控制（Traffic）

BaseAlgorithmModule 类

初始化与配置

`init(self)`

- 功能：初始化基础算法模块
- 详细内容：获取各种调度器的实例，包括计算调度器、通信调度器、实体调度器、奖励调度器等

`initialize(self, env: AirFogSimEnv, config={})`

- 功能：初始化算法与环境的交互
- 详细内容：
 - 设置奖励模型为负任务延迟-`task_delay`（即任务完成越快奖励越高）
 - 设置惩罚模型为固定值-1

核心调度函数

`scheduleStep(self, env: AirFogSimEnv)`

- 功能：执行一个完整的调度步骤

- 详细流程：依次调用各个子调度函数
 - a. 任务返回路由调度
 - b. 任务卸载调度
 - c. 通信资源调度
 - d. 计算资源调度
 - e. 任务分配调度
 - f. 交通控制调度

scheduleReturning(self, env: AirFogSimEnv)

- 功能：为计算完成的任务设置返回路径
- 详细流程：
 - a. 获取所有等待返回的任务
 - b. 对每个任务，计算其当前节点与所有RSU的距离
 - c. 选择最近的RSU作为返回路由
 - d. 设置任务的返回路径

scheduleMission(self, env: AirFogSimEnv)

- 功能：为任务分配合适的传感器和执行节点
- 详细流程：
 - a. 获取当前时间和待分配的任务配置
 - b. 对每个任务，根据其传感器类型和精度要求，在UAV上找到最低精度（但满足要求）的空闲传感器
 - c. 如果找到合适的传感器和节点，设置任务参数并生成对应的任务
 - d. 将任务添加到环境中
 - e. 从待分配列表中删除已处理的任务

scheduleTraffic(self, env: AirFogSimEnv)

- 功能：控制UAV的移动
- 详细流程：
 - a. 获取所有UAV的信息
 - b. 对每个UAV：
 - 尝试获取最近的任务位置
 - 如果有任务，计算移动方向和速度以接近目标位置

- 如果没有任务，随机生成移动方向和速度
- c. 设置所有UAV的移动模式

scheduleOffloading(self, env: AirFogSimEnv)

- 功能：决定任务卸载到哪个节点
- 详细流程：
 - a. 获取所有待卸载的任务
 - b. 对每个任务：
 - 获取任务节点的邻近节点（最多5个，按距离排序）
 - 选择最近的节点作为卸载目标
 - 设置任务的卸载目标

scheduleCommunication(self, env: AirFogSimEnv)

- 功能：为卸载任务分配通信资源
- 详细流程：
 - a. 获取可用的资源块数量和所有正在卸载的任务
 - b. 计算每个任务平均可分配的资源块数量
 - c. 为每个任务分配资源块
 - d. 设置通信资源分配结果

scheduleComputing(self, env: AirFogSimEnv)

- 功能：为任务分配计算资源
- 详细流程：
 - a. 定义CPU分配回调函数，该函数：
 - 收集所有计算任务并按节点分组
 - 对每个节点，平均分配其CPU资源给不超过3个任务
 - 返回分配结果
 - b. 设置计算资源分配回调函数

奖励计算函数

getRewardByTask(self, env: AirFogSimEnv)

- 功能：计算基于任务的奖励
- 详细流程：

1. 获取上一步成功和失败的任务
2. 对每个任务计算奖励并累加
3. 返回总奖励值

getRewardByMission(self, env: AirFogSimEnv)

- 功能：计算基于任务（mission）的奖励
- 详细流程：
 1. 获取上一步成功和失败的任务
 2. 分别计算成功任务的奖励和失败任务的惩罚
 3. 返回奖励、惩罚和总奖励

关于mission和task的区别

- Mission是更高层次的任务描述，可以包含多个Task
- Task是具体的计算单元，负责实际的数据处理

NVHAUAlgorithmModule 类

这是一个扩展算法，继承自BaseAlgorithmModule，实现了更复杂的调度策略。

关键特点与差异

scheduleMission(self, env: AirFogSimEnv)

- 功能扩展：
 - 按概率（50%）将任务分配给UAV或车辆
 - 当分配给UAV时，选择精度最低但满足要求的传感器
 - 当分配给车辆时，选择在感知范围内且精度满足要求的最近传感器

scheduleReturning(self, env: AirFogSimEnv)

- 功能扩展：
 - 根据当前节点类型选择不同的返回策略
 - 对于车辆节点：
 - 有50%概率选择通过UAV中继的方式返回（车辆→最近UAV→最近RSU）
 - 有50%概率选择直接返回（车辆→最近RSU）
 - 对于UAV节点：
 - 直接选择最近的RSU返回

其他函数

- scheduleTraffic、scheduleCommunication、scheduleComputing:
 - 直接继承基类实现
- scheduleOffloading:
 - 覆盖基类实现但未实现具体逻辑（pass）
- getRewardByTask、getRewardByMission:
 - 直接继承基类实现

核心算法逻辑

1. 任务生成与分配：将传感任务分配给合适的节点和传感器
2. 资源块分配：按任务平均分配通信资源
3. 计算资源分配：每个节点最多处理3个任务，平均分配CPU
4. 返回路由选择：基于概率选择直接或中继返回路径
5. UAV移动控制：优先移动到最近任务位置，无任务时随机移动